

Plataforma de monitoreo inteligente

Grupo Cordillera

Integrantes: Darlynee Zamorano

Profesora: Viviana Poblete

18/06/2026

Contextualización del caso

Grupo Cordillera enfrenta desafíos críticos en su sector retail:

- **Sistemas dispersos:** Gestión de ventas, inventario y atención operando de forma independiente.
- **Procesos ineficientes:** Generación manual de reportes, alta probabilidad de errores y datos no actualizados.
- **La Solución:** Implementación de una arquitectura de microservicios para centralizar datos, automatizar procesos y permitir decisiones en tiempo real.

Arquitectura Propuesta



Componentes Independientes

Cuatro servicios especializados: Usuarios, Ventas, Inventario y KPI. Cada uno con su propia base de datos (Database per Service).



API Gateway

Punto único de entrada centralizado para enrutamiento, autenticación (JWT) y seguridad global.

Stack Tecnológico

- **Backend:** Spring Boot (Java), Spring Cloud Gateway, FeignClient.
- **Frontend:** React (Arquitectura reactiva).
- **Base de Datos:** MySQL con persistencia JPA/Hibernate.
- **Mensajería:** RabbitMQ (Patrón Pub/Sub).

Estrategia de Comunicación

El sistema utiliza un enfoque híbrido para máxima eficiencia:

- **Síncrona:** Mediante OpenFeign para validaciones inmediatas (Usuarios/Productos).
- **Asíncrona:** Mediante RabbitMQ para eventos críticos (Notificaciones de stock, actualización de métricas KPI), permitiendo resiliencia.

Docker y Kubernetes

Docker

Empaquetado de cada microservicio con sus dependencias para asegurar consistencia en cualquier entorno.

Kubernetes

Orquestación automatizada para despliegue, escalado y monitoreo de los contenedores.

Frontend y Experiencia de Usuario

Desarrollado en React, con Control de Acceso Basado en Roles (RBAC):

- **Módulo Operativo:** Formulario optimizado para cajeros.
- **Módulo Analítico:** Paneles gerenciales con Recharts y Leaflet (geolocalización).
- **Servidor:** Nginx como proxy inverso para rendimiento y seguridad (CORS).

Aseguramiento de Calidad (Testing)

Proceso riguroso para certificar la estabilidad del sistema:

- **JUnit 5 y Mockito:** Pruebas unitarias de controladores y capas de servicio.
- **MockMvc:** Verificación de endpoints (201 Created vs 400 Bad Request).
- **RabbitTemplate:** Auditoría de la mensajería asíncrona mediante *verify()*.

Conclusión

El proyecto permitió integrar tecnologías modernas para resolver problemas reales de gestión y toma de decisiones.

Aprendizajes clave:

- Importancia del diseño arquitectónico previo.
- Complementariedad de Spring Boot, Docker y RabbitMQ.
- Visión integral: del código a las necesidades del negocio.

¿Preguntas?

Muchas gracias por su atención.